

# VIRTUALIZATION & CLOUD COMPUTING

## ONE-SHOT EXAM NOTES

5 PPTs combined • concise enough for ~1 hour revision • no syllabus topics intentionally removed

Source set: DS and CC, CC Models, CC Services, Virtualization, and Docker. Wording/terminology follows the uploaded PPTs; repeated material is consolidated rather than repeated.

### 0. HOW TO USE THESE NOTES

- First pass (30–35 min): read headings + bold concepts + comparison tables.
- Second pass (20–25 min): memorize definitions, advantages/disadvantages, classifications, and workflows.
- For 2–3 mark answers: definition + 3–4 precise points is usually the safest structure.
- For comparison questions: use a table or paired points; always mention the key differentiator.
- For diagrams: remember the layer/order and explain each component in one line.

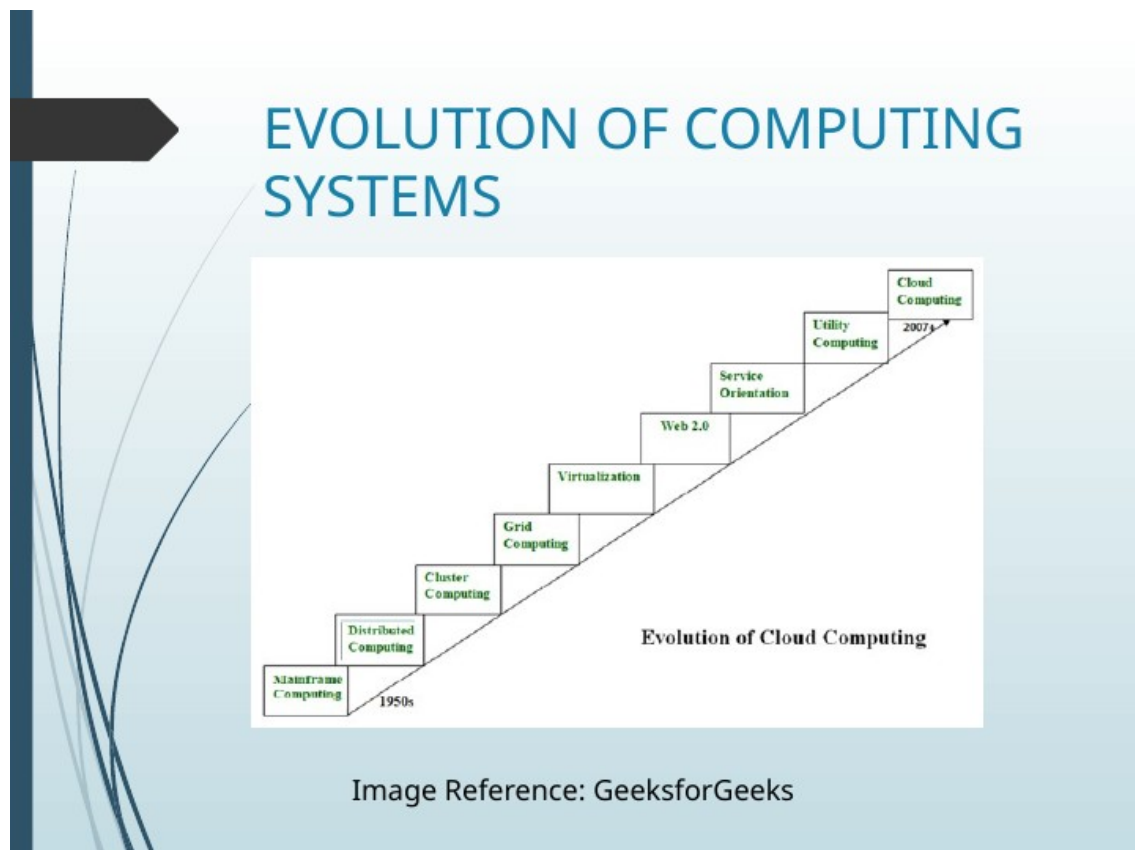
### 1. DISTRIBUTED COMPUTING & CLOUD FUNDAMENTALS

#### 1.1 Why study Cloud Computing?

- Backbone of modern IT systems.
- Enables scalability and global access.
- Foundation for AI, Big Data and IoT.
- Widely used in industry: AWS, Azure, GCP.

#### 1.2 Evolution of Computing Systems

PPT diagram: Evolution of Computing Systems



| Stage               | Period / idea | Key points                                                                                                                                                                                                      |
|---------------------|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mainframe           | 1950–1970     | Originated in 1951; powerful/reliable; large-scale data & I/O; bulk processing/online transactions; high fault tolerance and near-zero downtime; expensive.                                                     |
| Distributed systems | 1970–1980     | Multiple independent systems appear as one entity; resource sharing; scalability, concurrency, continuous availability, heterogeneity, independent failure handling.                                            |
| Cluster computing   | 1980s         | Multiple machines connected by high-bandwidth networks; alternative to mainframes; cost-effective; high computational performance; scalable by adding nodes; geographical restriction remained.                 |
| Grid computing      | 1990s         | Geographically distributed nodes over Internet; nodes may belong to multiple organizations; heterogeneous; solved location constraints but had bandwidth, latency and reliability issues; predecessor of cloud. |
| Utility computing   | —             | Pay-per-use model; compute, storage and infrastructure; resources provisioned on demand.                                                                                                                        |
| Virtualization      | 1972 onward   | IBM VM/370 released in 1972; virtual layer over hardware; multiple virtual instances on one machine; core cloud technology; used by platforms such as Amazon EC2 and VMware vCloud.                             |
| Web 2.0             | around 2004   | Interactive/dynamic web; interaction between users and cloud services; flexibility and collaboration; foundation of social media; examples: Google Maps, Facebook, Twitter.                                     |
| Service orientation | —             | Reference model for cloud; low-cost, flexible, scalable applications; introduces QoS, SLA and SaaS concepts.                                                                                                    |
| Cloud computing     | modern        | Internet-based, on-demand shared resources delivered as services.                                                                                                                                               |

### 1.3 Distributed Computing Principles

- Resource Sharing – efficient sharing of data, hardware and services among systems.
- Transparency – system appears as a single entity; hides underlying complexity.
- Scalability – ability to grow in users, resources and workload without losing performance.
- Fault Tolerance & Reliability – system continues operating despite individual component failures.
- Concurrency – multiple processes/users work simultaneously without conflict.

### 1.4 What is Cloud Computing?

- Internet-based computing where shared resources, software and information are provided to computers/devices on demand, similar to an electricity grid.
- Virtual shared servers provide software, infrastructure, platforms, devices and other resources/hosting on a pay-as-you-use basis.
- Data and applications can be stored on remote servers; reduces dependence on local hard drives/servers.
- Resources are delivered as services through the Internet; users need not have prior knowledge of managing the underlying resources.

### 1.5 Characteristics of Cloud Computing

PPT diagram: Five essential characteristics of cloud computing

## Characteristics of cloud computing

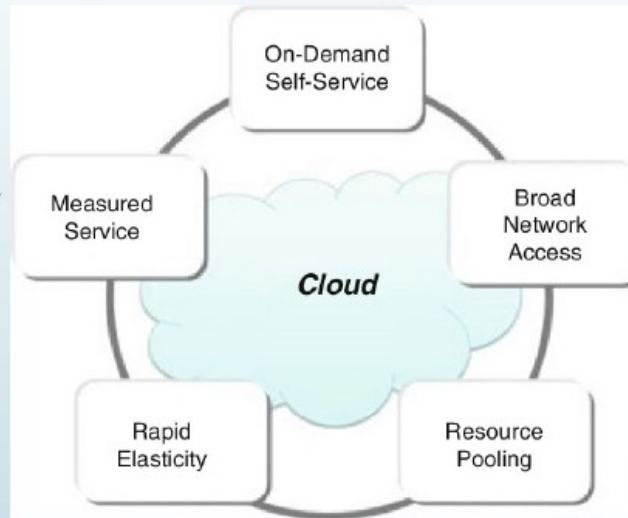


Image Credits: GeeksForGeeks

- On-Demand Self-Service – users obtain computing resources such as server time, storage and networking automatically, without provider human intervention.
- Broad Network Access – services are available over networks using standard mechanisms on laptops, phones, tablets and other devices.
- Resource Pooling – provider pools servers/storage for multiple customers (multi-tenancy); resources are dynamically assigned/reassigned as demand changes.
- Rapid Elasticity – resources can scale out/in quickly and appear almost unlimited to the consumer.
- Measured Service – usage of storage, processing, bandwidth etc. is monitored, controlled and optimized, enabling pay-per-use billing.

### 1.6 Benefits of Cloud Computing

- Cost efficiency – pay-as-you-use reduces capital and maintenance costs.
- Scalability & flexibility – resources scale up/down on demand.
- High availability & reliability – minimal downtime, backup and disaster recovery.
- Accessibility & mobility – access data/apps from anywhere through the Internet.
- Automatic updates & maintenance – provider handles infrastructure management.
- Efficient resource utilization – optimized use of computing and storage.

### 1.7 Challenges of Cloud Computing

- Security & privacy risks – off-premises data can face threats.
- Internet dependency – reliable/high-speed connectivity is required.
- Downtime & service outages – provider outages can affect availability.
- Limited control & vendor lock-in – dependence on one provider can restrict flexibility.
- Compliance & legal issues – data location and regulations can be difficult.
- Performance & latency – network delays may reduce application performance.

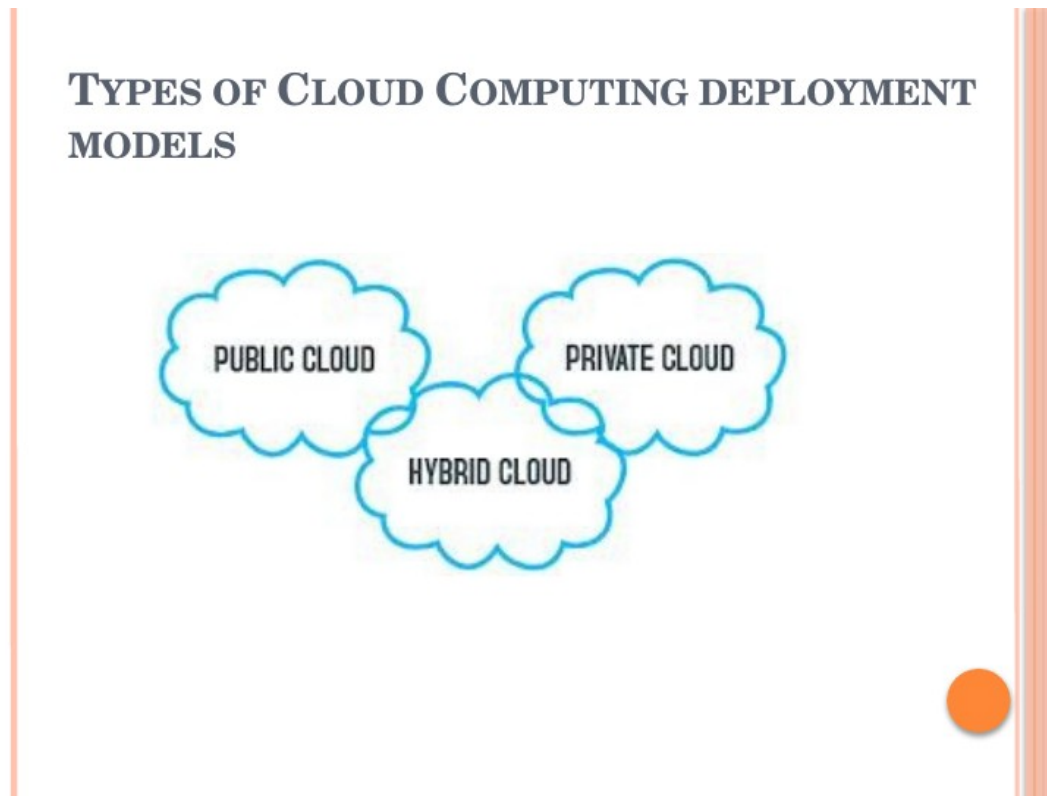
### 1.8 Business Drivers for Cloud Adoption

- Cost Optimization – shifts CapEx to OpEx through pay-as-you-go; reduces infrastructure/maintenance cost.
- Agility & Speed – rapid provisioning/scaling; faster deployment and market adaptation.
- Scalability & Elasticity – scale with demand and avoid over-provisioning during peaks.
- Innovation Enablement – access to AI, ML and Big Data technologies; focus on apps instead of infrastructure.

- Security & Compliance – advanced security, backup and disaster recovery capabilities.
- Digital Transformation – supports remote work, collaboration and data-driven decisions.
- Efficiency & Productivity – automation and simpler IT management frees teams for strategic work.
- Competitive Advantage – faster response to customers and quicker product/service launches.

## 2. CLOUD COMPUTING DEPLOYMENT MODELS

PPT diagram: Public, Private and Hybrid Cloud



*The deployment-model PPT covers four models: Public, Private, Hybrid and Community Cloud.*

### 2.1 Public Cloud

- Definition: cloud infrastructure made available to the general public by a service provider. Providers such as AWS, Microsoft and Google own/operate infrastructure and provide access over the Internet.
- Customers generally have no visibility/control over infrastructure location; customers share an infrastructure pool with limited configuration, security protections and availability variation.
- Benefits: pay-as-you-go; scalability; high availability; easy accessibility; no maintenance required; automatic updates.
- Disadvantages: limited control over information/infrastructure; privacy/integrity concerns; network performance depends on web/network speed; weaker security because resources are shared; limited customization.

### 2.2 Private Cloud

- Definition: cloud infrastructure dedicated to a particular organization; not shared with other organizations. Can be managed internally or by a third party and hosted internally or externally.
- Benefits: higher privacy/security; more control; improved reliability.
- Disadvantages – Cost: substantial on-premise build cost; personnel and periodic hardware upgrades; outsourced private cloud operating cost depends on resource usage/provider.
- Under-utilization: subscribed resources may be under-used, making optimization difficult.
- Capacity ceiling: physical hardware limits may restrict the amount of servers/storage supported.
- Vendor lock-in: outsourced infrastructure may force continued use of the same provider and make migration difficult.

### 2.3 Hybrid Cloud

- Definition: composition of two or more clouds (private, community or public) that remain unique entities but are bound together to provide advantages of multiple deployment models.

- Third-party cloud resources can be used fully/partially; public resources can augment private resources during unexpected workload surges.
- Benefits: risk management; flexibility; scalability; enhanced security; backup/disaster recovery; more opportunity for innovation.
- Disadvantages – Reliability: depends on technological and financial capability of cloud suppliers.
- Implementation: difficult to implement and maintain; requires strong network, storage and server capabilities; time-intensive and accuracy-sensitive.
- Hardware expenses: organization still maintains on-premise hardware for private-cloud needs; installation/maintenance costs are high.
- Security concern: combining public/private/on-premise environments introduces security risks, including third-party misuse and possible cyberattacks/breaches.

## 2.4 Community Cloud

- Definition: cloud infrastructure where systems/services are accessible by a group of organizations that share information and common considerations.
- Cost-effectiveness: can be cheaper than setting up separate private clouds; users share the same environment with logical session segmentation, reducing the need for separate servers.
- Regulatory compliance: shared interpretation/implementation of laws can reduce vulnerability to legal action.
- Industry-based security requirements: members can share expertise and technology for common security requirements.
- High availability: cloud solutions generally provide low downtime; the same applies to community clouds.
- Disadvantages: not suitable for every organization; slow data adoption; fixed storage/bandwidth shared among members; costlier than public cloud; sharing responsibilities is difficult.

## 2.5 Deployment Model Quick Comparison

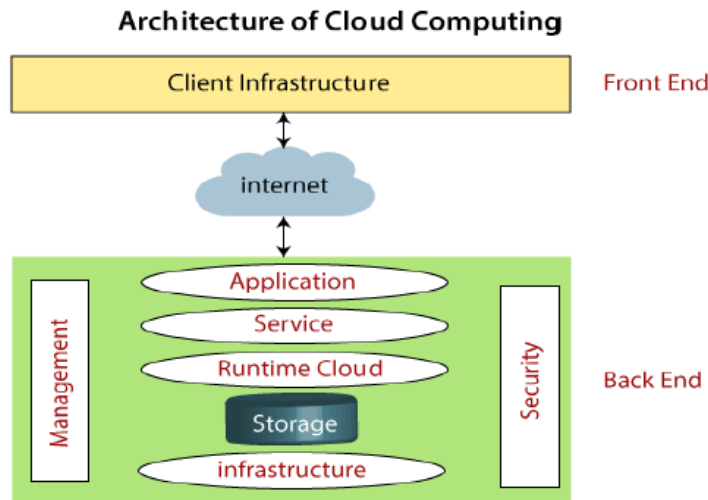
| Model     | Best remembered as                        | Main strengths                                              | Main weaknesses                                                         |
|-----------|-------------------------------------------|-------------------------------------------------------------|-------------------------------------------------------------------------|
| Public    | Shared provider infrastructure            | Low initial cost, scalable, accessible, provider-maintained | Less control/customization, shared-resource security concerns           |
| Private   | Dedicated to one organization             | Control, privacy/security, reliability                      | Cost, under-utilization, capacity ceiling, lock-in                      |
| Hybrid    | Combination of clouds                     | Flexibility, scalability, backup/DR, innovation             | Complex implementation, hardware cost, security/reliability concerns    |
| Community | Shared by organizations with common needs | Cost sharing, compliance, shared expertise, availability    | Shared resources/responsibility, limited fit, can cost more than public |

## 3. CLOUD COMPUTING ARCHITECTURE & SERVICE MODELS

### 3.1 Architecture: Front End vs Back End

PPT diagram: Cloud Computing Architecture

## CLOUD COMPUTING ARCHITECTURE



- Front End – used by the client; contains client-side interfaces/applications needed to access cloud platforms. Includes web browsers/servers such as Chrome, Firefox and Internet Explorer, thin/fat clients, tablets and mobile devices.
- Back End – used by the service provider; manages resources needed to provide cloud services, including storage, security mechanisms, virtual machines, deployment models, servers and traffic-control mechanisms.
- Internet – communication medium between front end and back end.

### 3.2 Main Architecture Components

1. Client Infrastructure – front-end component; provides GUI to interact with the cloud.
2. Application – any software/platform the client wants to access.
3. Service – manages which type of service is accessed according to client requirements.
4. Runtime Cloud – provides the execution/runtime environment to virtual machines.
5. Storage – provides large cloud storage capacity to store and manage data.
6. Infrastructure – host/application/network-level services; includes servers, storage, network devices, virtualization software and related resources.
7. Management – manages application, service, runtime, storage, infrastructure and security; coordinates the back-end components.
8. Security – built-in back-end component implementing security mechanisms.

### 3.3 SaaS – Software as a Service

- Also called cloud application services. Software is made available over the Internet, commonly through a subscription; usually runs directly in a browser without local installation.
- Examples in PPT: Google Apps, Salesforce, Dropbox, Slack, HubSpot, Cisco WebEx.
- Delivery: over the Internet.
- Advantages: no need to install/run software locally; available whenever logged in; accessible from different devices with Internet; personalized user logins/access levels; provider handles maintenance/compliance/security; subscription structure.
- Characteristics: Internet-accessible; hosted on remote third-party servers; scalable tiers for small/medium/enterprise businesses; security, compliance and maintenance included in cost.
- When to use: useful where the organization wants software such as email to work without managing updates/settings. Provider handles problems and maintenance.

### 3.4 PaaS – Platform as a Service

- Also called cloud platform services. Provides a platform for software creation; unlike SaaS, the user/developer uses the platform to build applications.
- Examples in PPT: Windows Azure, Force.com, Magento Commerce Cloud, OpenShift.
- PaaS vendor provides hardware/software tools over the Internet; typical users are developers.
- Advantages: developers do not start from scratch; saves time and money; avoids many low-level management tasks.
- Characteristics: accessible by multiple users; scalable resource tiers; built on virtualization technology; can be run without extensive system-administration knowledge.
- When to use: cost/time-effective for creating a unique application; lets developers focus on creating, testing and deploying rather than updates/security patches.

### 3.5 IaaS – Infrastructure as a Service

- Also called cloud infrastructure services. Provides cloud-based alternatives to on-premise infrastructure and supports management of applications/data/middleware/runtime environments.
- Examples in PPT: AWS EC2, Google Compute Engine (GCE), Cisco Metapod.
- Services include pay-as-you-go storage, networking and virtualization.
- Delivery: over the Internet.
- Advantages: avoids large investment in physical on-site resources and labor-intensive maintenance; highly flexible and scalable; resources can be replaced/changed without losing initial on-premise investment.
- Characteristics: highly flexible; highly scalable; accessible by multiple users; cost-effective.

### 3.6 On-Premises vs IaaS vs PaaS vs SaaS

PPT comparison diagram: who manages each layer



| Layer          | On-Premises | IaaS             | PaaS             | SaaS             |
|----------------|-------------|------------------|------------------|------------------|
| Applications   | You manage  | You manage       | You manage       | Provider manages |
| Data           | You manage  | You manage       | You manage       | Provider manages |
| Runtime        | You manage  | You manage       | Provider manages | Provider manages |
| Middleware     | You manage  | You manage       | Provider manages | Provider manages |
| OS             | You manage  | You manage       | Provider manages | Provider manages |
| Virtualization | You manage  | Provider manages | Provider manages | Provider manages |
| Servers        | You manage  | Provider manages | Provider manages | Provider manages |

|            |            |                  |                  |                  |
|------------|------------|------------------|------------------|------------------|
| Storage    | You manage | Provider manages | Provider manages | Provider manages |
| Networking | You manage | Provider manages | Provider manages | Provider manages |

*Memory trick: as you move from On-Premises → IaaS → PaaS → SaaS, the provider manages more layers and you manage less.*

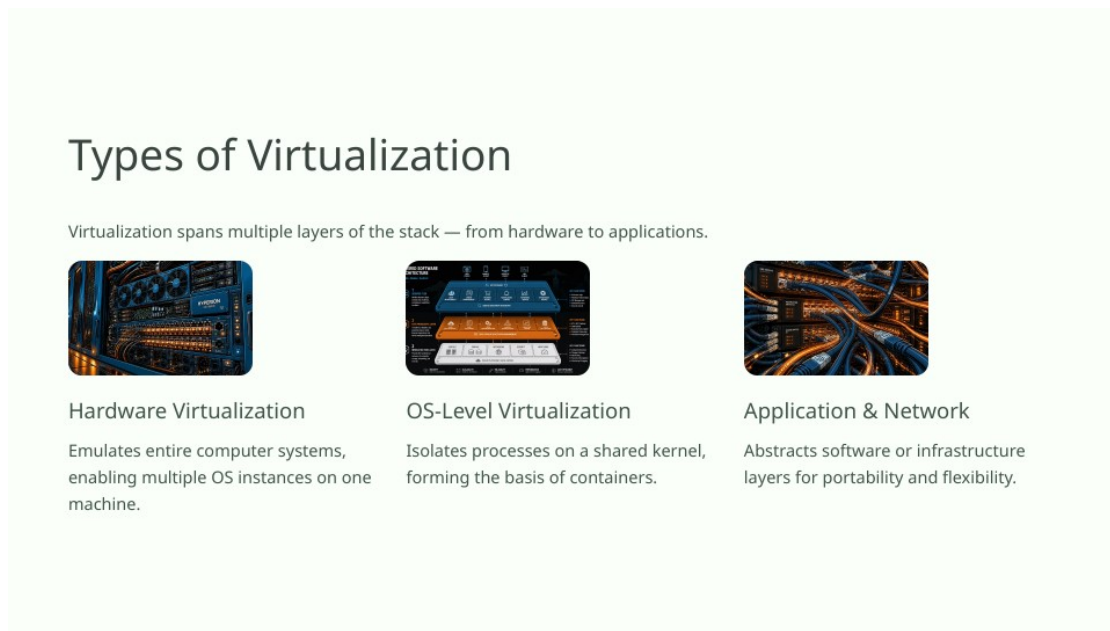
## 4. VIRTUALIZATION & CONTAINERIZATION

### 4.1 Definition and Core Goals

- Virtualization decouples resources from their physical providers, allowing multiple isolated environments to share a single host.
- Abstraction – separates software from hardware; creates virtual CPU, memory, storage and networking.
- Isolation – each virtual environment runs independently without interference between guests.
- Consolidation – multiple workloads share one physical host, reducing cost and complexity.

### 4.2 Types of Virtualization

**PPT diagram: Types of Virtualization**



- Hardware virtualization – emulates entire computer systems, enabling multiple OS instances on one machine.
- OS-level virtualization – isolates processes on a shared host kernel; foundation of containers.
- Application & network virtualization – abstracts software/infrastructure layers for portability and flexibility.

### 4.3 Hardware Virtualization

- Transforms one physical server into multiple isolated guest machines, each with its own kernel and OS.
- Server consolidation – run many workloads on fewer physical servers.
- Multi-platform testing – test different OS configurations without dedicated hardware.

### 4.4 Full Virtualization vs Paravirtualization

**PPT comparison: Full vs Paravirtualization**

## Full vs. Paravirtualization

### Full Virtualization

The VMM (virtual machine monitor) presents an abstraction **identical to physical hardware**. Guest OS requires **no modifications** — it runs unmodified, unaware of the hypervisor.

- Complete hardware emulation
- Higher overhead due to binary translation
- Examples: VMware, QEMU

### Paravirtualization

The guest OS is **modified to be hypervisor-aware**, making explicit calls that reduce overhead and boost performance.

- Guest OS modified for the hypervisor
- Lower overhead, higher throughput
- Examples: Xen (paravirt mode)

| Feature              | Full Virtualization                     | Paravirtualization                       |
|----------------------|-----------------------------------------|------------------------------------------|
| Guest OS             | Unmodified; unaware of hypervisor       | Modified to be hypervisor-aware          |
| Hardware abstraction | Identical/complete hardware abstraction | Guest makes explicit calls to hypervisor |
| Performance          | Higher overhead; binary translation     | Lower overhead; higher throughput        |
| Examples             | VMware, QEMU                            | Xen (paravirtualization mode)            |

## 4.5 Hypervisors / VMM

### PPT diagram: Type 1 vs Type 2 Hypervisors

## Hypervisors: The Virtual Machine Monitor

The hypervisor (VMM) sits between hardware and VMs, managing **resource allocation, isolation, and lifecycle**.



### Type 1 — Bare-Metal

Runs **directly on hardware** with no host OS. Ideal for production

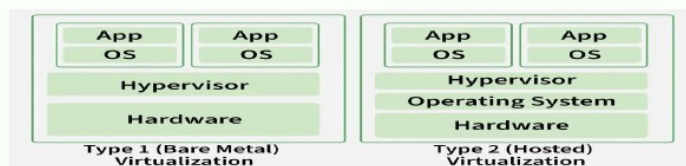
- Xen, VMware ESXi, Microsoft Hyper-V



### Type 2 — Hosted

Runs **within a conventional host OS**. Great for development.

- VirtualBox, VMware Workstation



- Hypervisor (Virtual Machine Monitor/VMM) sits between hardware and VMs; manages resource allocation, isolation and VM lifecycle.
- Type 1 – Bare-metal: runs directly on hardware without a host OS; suited for production. Examples: Xen, VMware ESXi, Microsoft Hyper-V.
- Type 2 – Hosted: runs inside a conventional host OS; useful for development. Examples: VirtualBox, VMware Workstation.

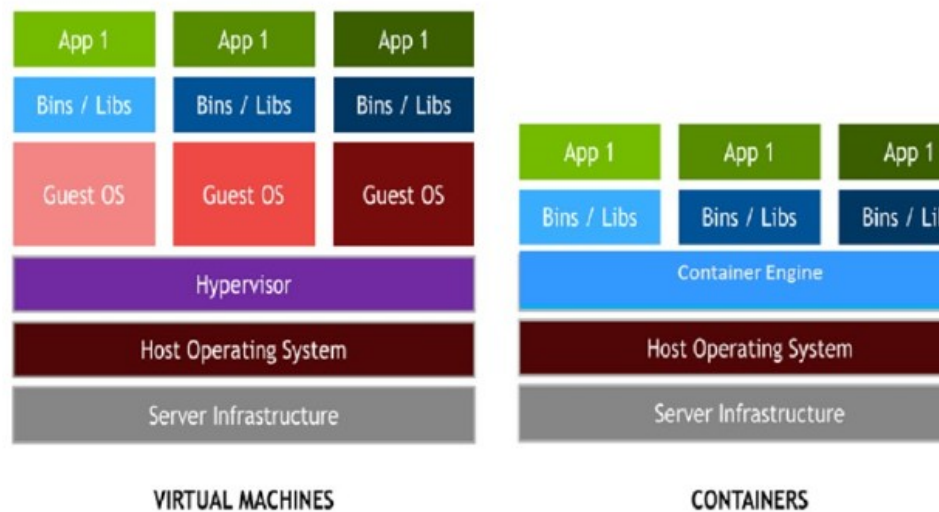
## 4.6 Hardware-Assisted Virtualization

- Intel VT-x and AMD-V provide CPU-level instructions that handle virtualization in hardware and reduce hypervisor overhead.
- Simplified transitions – hardware directly manages guest-to-host mode switches.
- Reduced binary translation – privileged instructions can execute natively instead of costly emulation.
- Near-native performance – VMs can approach bare-metal speed for compute-intensive workloads.

## 4.7 OS-Level Virtualization / Containers

PPT diagram: Virtual Machines vs Containers

# Virtualization vs Containerization



- Unlike hardware VMs, containers share the host kernel, making them much lighter and faster.
- Lightweight – no duplicate kernels; startup in seconds.
- Efficient – more instances per host than VMs.
- Isolated – processes are isolated and should not interfere across containers.
- Containers are the foundation of modern cloud-native architectures.

## 4.8 Application Virtualization

- Decouples software from the underlying OS so apps can run in self-contained packages without installation dependencies.
- No installation required – apps run in isolated sandboxes without modifying the host OS.
- Portability – same package can run across different OS versions/environments.
- Legacy support – outdated software can run on modern systems without compatibility problems.

## 4.9 Virtual Machines vs Containers

| Aspect         | Virtual Machines                  | Containers                      |
|----------------|-----------------------------------|---------------------------------|
| OS             | Each VM has a guest OS/kernel     | Share host OS kernel            |
| Virtualization | Hardware-level via hypervisor     | OS-level                        |
| Overhead       | Higher                            | Lower                           |
| Startup        | Slower                            | Seconds / fast                  |
| Density        | Fewer instances per host          | More instances per host         |
| Typical use    | Strong OS isolation, multiple OSs | Cloud-native apps/microservices |

The Docker PPT diagram visually shows VMs as App + Bins/Libs + Guest OS over a Hypervisor, while containers use App + Bins/Libs over a Container Engine and share the Host OS.

## 5. DOCKER & CONTAINERIZATION

### 5.1 Problem Without Docker

- Scenario: Student Management Web Application using Python (Flask) and MySQL; goal is deployment from development to production.
- Development: Windows + Python 3.10 + specific library versions; works locally → “It works on my machine!”
- Testing: different system (Linux + Python 3.8) → crashes, dependency/version mismatch, different behavior.
- Deployment: missing libraries, port conflicts, manual installation; deployment may take hours/days.
- Scaling/maintenance: difficult to replicate setup; slow/error-prone scaling; updates may break existing apps; rollback is difficult.
- Core problems: environment inconsistency, dependency conflicts, slow deployment, poor scalability; application depends on the environment, not just the code.

### 5.2 Docker Solution

- Developer packages the application + dependencies into a Docker Image; environment becomes fixed and consistent.
- Tester runs the same image, so application behavior remains consistent.
- The same image can run on servers or cloud; scaling becomes fast and reliable.
- Key phrase: Build once, run everywhere.

### 5.3 Why Docker Solves the Problem

- Same environment across development, testing and production.
- No dependency/version mismatch.
- Fast deployment and easy rollback.
- Efficient use of system resources.
- Supports modern microservices architecture.

### 5.4 Before Docker vs After Docker

| Before Docker                  | After Docker                           |
|--------------------------------|----------------------------------------|
| Environment mismatch           | Consistent environments                |
| Manual dependency installation | Application packaged with dependencies |
| Slow deployment                | Fast/automated deployment              |
| Hard to scale                  | Easy scaling                           |
| High risk of failure           | Reliable and repeatable                |

### 5.5 What is Docker?

- Docker is an open platform for developing, shipping and running applications.
- Separates applications from infrastructure, enabling faster software delivery.
- Reduces the gap between development code and production deployment.

### 5.6 Containers in Docker

- Containers are lightweight, isolated environments in which applications run.
- A container includes application code, dependencies and runtime environment.
- The same containerized application can run consistently across development, testing and production.

### 5.7 Docker Platform Capabilities

- Build applications using containers.
- Use containers as the unit of testing and distribution.
- Deploy on local servers, cloud or hybrid environments.
- Manage the entire container lifecycle efficiently.

### 5.8 Docker Architecture

**PPT diagram: Docker Architecture**

# Docker Architecture

The diagram illustrates the Docker Architecture, showing the interaction between various components:

- Docker Engine:** The central component, containing the Docker Client and Docker Daemon.
- Docker Client:** Interacts with the Docker Engine via the REST API.
- Docker Host:** The environment where containers run, containing the Docker Daemon and Containers.
- Docker Daemon:** Manages the Docker Host's state and interacts with the Docker Client via the REST API.
- Containers:** The application environments running on the Docker Host.
- Docker Registry:** A central repository for storing and distributing Docker images.
- Images:** The templates used to create containers, stored in the Docker Registry.

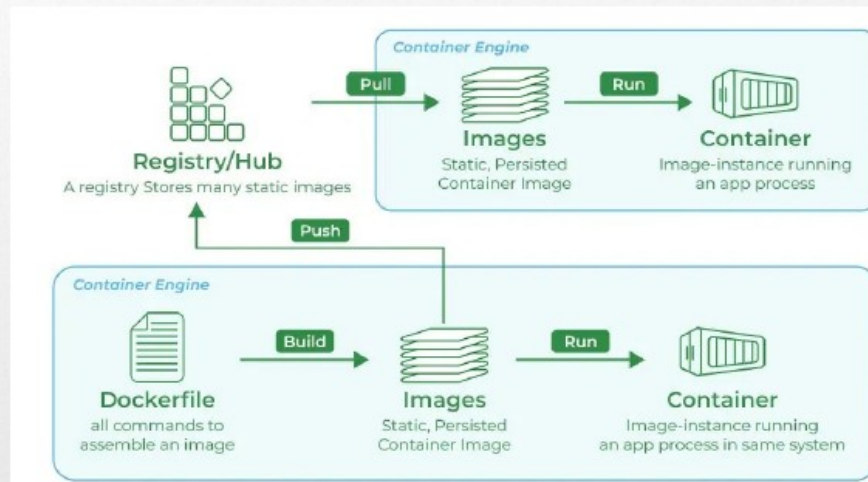
The diagram shows the flow of data and control between these components, highlighting the REST API as the primary communication channel.

- Architecture flow to remember: Docker Client → REST API → Docker Daemon → Images/Containers/Networks/Volumes; Registry stores/distributes images.

## 5.9 Docker Hub

### PPT diagram: Docker Hub image workflow

# Docker Hub

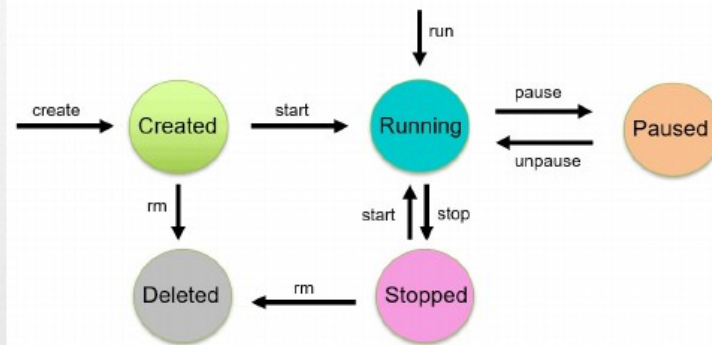


- Docker Hub acts as a registry/hub that stores Docker images.
- Workflow shown in PPT: Dockerfile → build → Image → run → Container; Image can be pushed to a registry and later pulled from the registry → run → Container.

## 5.10 Container Lifecycle

PPT diagram: Container Lifecycle

# Container lifecycle



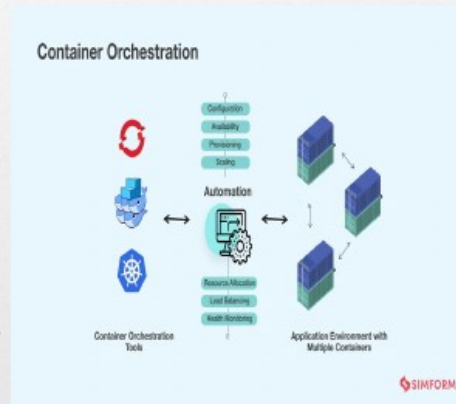
- Created → Running → Stopped → Deleted.
- Created → start → Running.
- Running → stop → Stopped; Stopped → start → Running.
- Running → pause → Paused; Paused → unpause → Running.
- Created → rm → Deleted; Stopped → rm → Deleted.
- The lifecycle diagram in the PPT specifically uses the commands/actions: create, start, run, pause, unpause, stop and rm.

## 5.11 Container Orchestration

PPT diagram: Container Orchestration

# Container Orchestration

- Container orchestration automates the deployment, scaling, and management of containerized applications.
- It handles the lifecycle of containers — from provisioning to scaling and monitoring.



- Definition: automation of deployment, scaling and management of containerized applications.
- Handles container lifecycle from provisioning to scaling and monitoring.
- Without orchestration: manual container management becomes error-prone at large scale; difficult to track host utilization, perform updates and handle failures.
- Orchestration helps by automating container tasks, using resources efficiently, and simplifying deployment, scaling and updates.

## 5.12 Benefits of Container Orchestration

- Efficient Resource Management – optimizes CPU, memory and storage.
- Cost Savings – reduces human labor and infrastructure waste.
- Greater Agility – faster deployment cycles.
- Scalability – applications can be scaled up or down easily.
- Better Security & Governance – isolation and fewer configuration errors.

## 6. LAST-MINUTE HIGH-YIELD DEFINITIONS

**Cloud Computing** — Internet-based, on-demand delivery of shared computing resources/services.

**Distributed Computing** — Multiple independent systems working together and appearing as a unified system.

**Virtualization** — Abstraction of physical resources into isolated virtual environments.

**Hypervisor/VMM** — Software layer that manages virtual machines and allocates physical resources.

**Container** — Lightweight isolated environment sharing the host kernel and packaging application + dependencies.

**Docker** — Platform for developing, shipping and running applications using containers.

**SaaS** — Ready-to-use software delivered over the Internet.

**PaaS** — Cloud platform/tools used mainly by developers to build applications.

**IaaS** — Cloud-based compute/storage/networking/virtualization infrastructure.

**Public Cloud** — Provider-owned shared cloud offered to the general public.

**Private Cloud** — Cloud dedicated to one organization.

**Hybrid Cloud** — Combination of two or more distinct cloud environments bound together.

**Community Cloud** — Shared cloud for organizations with common requirements.

## 7. 10 THINGS NOT TO MIX UP

- Public vs Private: shared provider infrastructure vs dedicated organization infrastructure.
- Hybrid vs Community: hybrid combines different cloud types; community is shared by organizations with common concerns.
- IaaS vs PaaS vs SaaS: infrastructure → development platform → ready-to-use software.
- VM vs Container: guest OS + hypervisor vs shared host kernel + container engine.
- Full vs Para virtualization: unmodified guest OS vs hypervisor-aware modified guest OS.
- Type 1 vs Type 2 hypervisor: bare-metal vs hosted inside a host OS.
- Image vs Container: image is the packaged/static template; container is a running instance/process from the image.
- Docker Client vs Daemon: client sends commands; daemon performs/manages Docker operations.
- Docker Hub/Registry vs Docker Engine: registry stores/distributes images; engine runs/builds/manages containers.
- Rapid Elasticity vs Scalability: rapid elasticity emphasizes quick automatic scale in/out; scalability is the ability to grow/shrink with demand.

*Exam-safe answer pattern: Definition → key working/architecture → 3–5 points → example (if given in PPT) → advantage/disadvantage where relevant.*